

Lab 9: Priority Queues and Heaps

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** PQ ADT, comparators, array-based heap, heap sort

Objectives

- Define the Priority Queue ADT with **insert** and **removeMin**.
- Use Java's **Comparator<E>** interface for flexible priority orderings.
- Implement a binary min-heap backed by an **ArrayList**, using up-heap and down-heap.
- Apply the heap to implement heap sort in $O(n \log n)$ time.

Quick Reference

Priority Queue interface.

```
int size(); boolean isEmpty();
Entry<K,V> insert(K key, V value);
Entry<K,V> min();           // peek
Entry<K,V> removeMin();
```

Heap property (min-heap). $\text{key}(\text{parent}) \leq \text{key}(\text{child})$ for every node.

Array-based heap (1-indexed, index 0 unused).

Parent of i : $\lfloor i/2 \rfloor$. Left child: $2i$. Right child: $2i + 1$.

insert / up-heap. Add at end; swap with parent while smaller than parent.

removeMin / down-heap. Replace root with last element; swap with smaller child.

Complexity. insert: $O(\log n)$. removeMin: $O(\log n)$. min: $O(1)$.

Quick Experiments (Try It)

Try It 1: Heap shape property

Insert keys 5, 3, 8, 1, 4, 7, 2 one by one. After each insert, print the backing array (index 1 to size). Verify it is always a valid min-heap.

Try It 2: Custom comparator for max-heap

Use $(a, b) \rightarrow b.\text{compareTo}(a)$ as the comparator. Insert the same keys and call **removeMin()** repeatedly. What order do the keys come out?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Implement HeapPriorityQueue<K,V> (55 min)

Back by an **ArrayList<Entry<K,V>** (1-indexed; dummy entry at index 0).

- **upheap(int j):** swap with parent while heap property violated.
- **downheap(int j):** swap with smaller child while violated.
- **insert(K key, V value):** add at end, call **upheap**.
- **removeMin():** swap root with last, remove last, call **downheap(1)**.
- Accept a **Comparator<K>** (default to natural order if null).

Test: insert 8 entries, call **min()**, then **removeMin()** 3 times.

In-Lab Exercise 2: Priority Queue Sort (20 min)

Implement `void pqSort(List<Integer> list)`: insert all into your heap, then remove with `removeMin` and write back. Verify sorted order. Compare time complexity with selection sort ($O(n^2)$).

In-Lab Exercise 3: In-Place Heap Sort (30 min)

Implement `void heapSort(int[] a)` in-place using a max-heap. Phase 1: heapify in $O(n)$ via `downheap` from $\lfloor n/2 \rfloor$ down to 1. Phase 2: repeatedly swap `a[1]` (max) with last unsorted element; call `downheap`. Test on random, sorted, and reverse-sorted arrays of size 15.

In-Lab Exercise 4: k Largest Elements (15 min)

Write `List<Integer> kLargest(int[] a, int k)`: maintain a min-heap of size k . For each element: if heap size $< k$, insert; else if element $>$ heap min, replace. Test with $k = 3$ on an array of 10 integers.

AI Collaboration**Suggested prompts:**

- “Trace the upheap step when inserting key 2 into the heap [1,4,3,7,6,5].”
- “Why does building a heap bottom-up run in $O(n)$ rather than $O(n \log n)$?”

Verify: Ask the AI for the heap array after inserting keys 5, 3, 8, 1. Run your implementation and compare.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Heap insert trace

Insert keys 9, 4, 7, 2, 6, 1 into an empty heap. After each insert, write the backing array (positions 1 to size). Draw the complete binary tree after the final insert.

Trace & Predict 2: removeMin trace

From heap $[-, 1, 4, 3, 7, 6, 8]$, trace `removeMin()`. Show the array after the root-last swap, then after each swap in `downheap`.

Write Code on Paper 1: Merge k sorted lists

Given k sorted `int[]` arrays, write a method to merge them using a min-heap. Hint: insert the first element of each array as `(value, arrayIndex, elementIndex)`; each time you extract the min, insert the next element from the same array. State the time complexity in terms of n (total elements) and k .

Submission. Save files as `lab9_ex*.java` and submit on the course site.